

RideFleet — roteiro de apresentação dos requisitos

grupo-11 · requisito → arquivo:linha → o que explicar ao professor · caminhos relativos à pasta ridefleet2/

1. Webhooks (contrato com o Core)

Requisito	Onde	O que explicar
Receber convite do leilão e responder proposta síncrona	<code>backend/routes/rides.py:319</code> (<code>receber_leilao</code>)	Core faz POST <code>/rides/incoming</code> ; respondemos 200 com <code>estimatedEta</code> , <code>estimatedPrice</code> e <code>logicalTimestamp</code> na própria resposta HTTP
Cálculo da proposta	<code>rides.py:309</code> (<code>_montar_proposta</code>) · <code>:296</code> (<code>haversine</code>) · <code>:264</code> (<code>constantes</code>)	ETA cai 30s por motorista livre extra (piso 120s); preço = bandeirada R\$ 5,00 + R\$ 2,50/km
Respeitar <code>auctionDeadline</code>	<code>rides.py:334</code>	Convite com prazo vencido é recusado com 204 — a proposta seria ignorada pelo Core
Recusa sem penalidade / nunca 5xx	<code>rides.py:350</code> (<code>except</code> → 204)	Timeout e HTTP ≥ 500 alimentam o circuit breaker do Core contra o grupo; erro interno vira recusa silenciosa
Callback do vencedor	<code>rides.py:370</code> (<code>corrida_atribuida</code>)	Core faz POST <code>/rides/{uuid}/assigned</code> só no grupo vencedor; confirmamos o recebimento com 200
Lock transferido + <code>lockExpiresAt</code>	<code>rides.py:417-424</code>	O lock distribuído chega transferido com o callback; renovamos via POST <code>/locks/{uuid}</code> para ganhar folga — se a renovação falhar mas o lock transferido ainda valer, seguimos
Transição match → confirm dentro do lock	<code>rides.py:426-429</code>	PATCH <code>/rides/{uuid}/status</code> no Core, com 1 retry re-renovando o lock
Falha → compensação + re-leilão	<code>rides.py:355</code> (<code>_compensar_aceite_local</code>)	Libera o motorista, cancela a corrida local e responde 409 SEM liberar o lock: a expiração é o gatilho para o Core compensar e re-leiloar excluindo o grupo
Idempotência (retry do Core)	<code>rides.py:386</code>	Callback repetido devolve a mesma <code>corrida_id</code> — não duplica corrida nem ocupa segundo motorista
Relógio de Lamport	<code>core_client.py:33</code> (<code>_tick</code>)	$\max(\text{local}, \text{remoto}) + 1$ a cada mensagem; webhooks aplicam via <code>_tick_seguro</code> (tolerante a payload malformatado)

Seleção do vencedor é executada NO CORE (`ridefleet-core-sin142/app/workers/auction_worker.py:50`): menor preço → menor ETA → groupId alfabético.

2. Comunicação com o Core (lado cliente)

Requisito	Onde	O que explicar
Registro do grupo + API key	<code>core_client.py:58</code> · boot em <code>app.py:70</code>	POST <code>/groups/register</code> na inicialização; a <code>apiKey</code> retornada vai no header X-API-Key de toda chamada ao Core
Delegação de saída (leilão)	<code>rides.py:178</code> (<code>solicitar</code>) → <code>:245</code> (<code>_delegar_via_core</code>) → <code>core_client.py:84</code>	Sem motorista ou congestionado (<code>fila.py:38</code>): corrida entra na fila RabbitMQ e POST <code>/rides</code> abre o leilão no Core
Locks distribuídos	<code>core_client.py:138</code> (<code>adquirir/renovar</code>) · <code>:151</code> (<code>liberar</code>)	POST <code>/locks/{uuid}</code> com TTL; liberação explícita ao concluir a corrida
Avanço da saga	<code>core_client.py:166</code>	PATCH <code>/rides/{uuid}/status</code> — estados: <code>confirm</code> , <code>in_transit</code> , <code>complete</code> , <code>cancelled</code>
Timeout de corrida delegada (500s)	<code>rides.py:84</code> (<code>config</code>) · <code>:91</code> (<code>passo do vigia</code>) · <code>:131</code> (<code>thread</code>) · <code>:148</code> (<code>boot</code>)	Core não chama nossos webhooks para corridas que NÓS enviamos (excluídos do próprio leilão); o vigia consulta o status a cada 20s, espelha o estado real e cancela (local + Core) se não concluir em <code>DELEGACAO_TIMEOUT_SEGUNDOS</code>

3. Observabilidade (3,0 pts)

Métrica exigida	Exposta em	Alimentada por
Endpoint padrão Prometheus	<code>app.py:199 (/metrics)</code>	Formato texto OpenMetrics, scrapeável pelo Prometheus do Core
Corridas locais vs. delegadas <code>ridefleet_rides_local_total</code> · <code>ridefleet_rides_delegated_total</code>	<code>app.py:285</code> e <code>:289</code>	Contadores em <code>metrics.py</code> , incrementados em <code>rides.py</code>
Recebidas de outros grupos <code>ridefleet_rides_received_total</code>	<code>app.py:293</code>	<code>registrar_corrida_recebida()</code> chamado no webhook <code>/assigned</code> (<code>rides.py:435</code>)
Latência dos endpoints de corrida <code>ridefleet_rides_latencia_mediana_ms</code> · <code>_p95_ms</code>	<code>app.py:236-243</code>	Coleta por rota em <code>app.py:31 (_registrar_latencia)</code>
Throughput (req/s) <code>ridefleet_throughput_rps</code>	<code>app.py:254-256</code>	Janela móvel de 60s em <code>app.py:62 (_throughput_rps)</code>
Estado do serviço <code>ridefleet_service_state</code>	<code>app.py:259-261</code>	0=disponível, 1=congestionado, 2=indisponível — mesma régua do <code>/health</code> (<code>app.py:70</code>)
Filas de entrada e saída <code>ridefleet_fila_entrada</code> · <code>ridefleet_fila_saida</code>	<code>app.py:218+</code>	Contagem real no RabbitMQ via <code>fila.py:102 (tamanho_fila)</code>
Distribuição entre instâncias <code>label instance_id</code>	<code>app.py:20</code>	Env <code>INSTANCE_ID</code> no <code>docker-compose.yml:46/79</code> ; Prometheus do grupo scrapeia <code>backend1/backend2</code> separados

Grafana do Core: o Prometheus do Core ganha um job por grupo (template comentado no `prometheus.yml` dele) apontando para `10.226.0.105:5000` — a partir daí qualquer painel/Explore enxerga `ridefleet_{service="grupo-11"}`. Verificado em 12/06 numa réplica local do ambiente do professor: Grafana retornou `rides_delegated_total=1` após delegação real. Testes: `tests/test_contrato.py` (`test_metrics_observabilidade_completa`).

4. CI/CD (2,0 pts) — `.github/workflows/ci.yml`

Estágio	Linha	O que faz
build	<code>ci.yml:14</code>	docker build da imagem do backend
testes unitários	<code>ci.yml:25</code>	pytest (exceto contrato), com relatório de cobertura
testes de contrato do Core	<code>ci.yml:51</code>	<code>tests/test_contrato.py</code> — simula exatamente as chamadas do Core (webhooks + métricas)
teste de integração	<code>ci.yml:78</code>	docker compose up da stack inteira no runner + smoke via load balancer: <code>/health</code> , métricas, <code>/rides/incoming</code> (200 + proposta) e <code>/assigned</code> sem Core (409 + compensação)
deploy automático	<code>ci.yml:146</code>	Só em push na main: publica a imagem no GHCR com tags <code>:latest</code> e <code>:</code>

Encadeamento: cada estágio só roda se o anterior passou (needs:). Pull request executa tudo, menos o deploy.

5. Extras que podem ser perguntados

Item	Onde
Load balancer (least_conn, retry automático, keepalive)	<code>nginx.conf:24</code>
Política de overflow / congestionamento	<code>fila.py:38 (esta_congestionado)</code>
Logs estruturados JSON (auditoria causal)	<code>logger.py</code>
Health check UP/DEGRADED/DOWN	<code>app.py:171</code>
Diagnóstico da conexão com o Core	<code>app.py:142 (/debug/core)</code>
Painel admin restrito (backend + frontend)	<code>routes/admin.py:10 (guard)</code> · <code>routes/auth.py:12/52 (flag_is_admin)</code> · <code>ridefleet-frontend/src/routes/AdminRoute.jsx</code> · <code>Navbar.jsx</code> (link condicional)

Item	Onde
Suíte de testes (54) — rodar ao vivo impressiona	<code>backend/tests/ · python -m pytest tests/ -v</code>